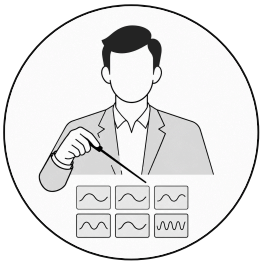
Agent Team

Five AI agents under one supervisor built the Size-Level Availability Processor in nine waves. Every task follows the same cycle. Agents pass work to each other through files, never through chat memory.

5 agents | 1 supervisor | 9 waves | 6-step task cycle | Claude Haiku 5.5

Team structure



Sam

SUPERVISOR · LEADS EVERY WAVE



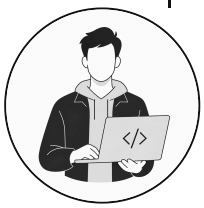
Poppy

PRODUCT OWNER



Tom

QA ENGINEER



Dan

SOFTWARE ENGINEER



Max

MANAGER

How it is set up

01

Source of truth

Assignment, plan overview and the wave briefs in `plan/`.

02

Team rules

Roster, shared rules and work folder layout in `.github/copilot-instructions.md`.

03

Personas

One agent file per role in `.github/agents/`: model, tools, limits.

04

Process

The `task-cycle` and `service-docs` skills, plus the code style.

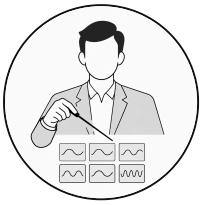
05

Hand-off

Files in `work/wave-NN/` replace chat memory between agents.

Who does what

Each agent has one file in `.github/agents/`. The file sets its model, its tools and its limits. Only Sam starts the other four.



Sam SUPERVISOR · ENTRY POINT

Leads the waves end to end. Runs the waves and the tasks in order, starts the other agents, gates each task on QA and closes each wave. Coordinates; does not implement.

WRITES	TOOLS	NEVER
Session todos, gate decisions, wave folders	`read, search, edit, agent, todo`	Implements or tests. Settles source conflicts silently.



Poppy PRODUCT OWNER

Turns the assignment and the wave brief into clear requirements and test expectations for each task. Owns the requirements, not the code or the tests.

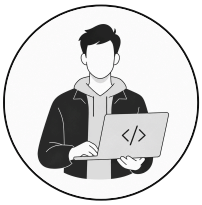
WRITES	TOOLS	NEVER
`work/wave-NN/requirements.md`	`read, search, edit`	Code, tests or implementation advice. Every requirement cites its source.



Tom QA ENGINEER · QUALITY GATE

Owns the tests and the quality gate. Writes the tests before the code, reviews the diff, runs manual tests against WireMock and gives the PASS or FAIL verdict.

WRITES	TOOLS	NEVER
`test/` and `work/wave-NN/qa.md`	`read, search, edit, execute`	Production code. PASS only when everything is green.



Dan SOFTWARE ENGINEER · BUILDER

Implements one task at a time, once Tom's tests exist, until they pass. Runs the verification commands and writes the hand-off. In the documentation wave he writes the service docs.

WRITES	TOOLS	NEVER
`src/`, `work/wave-NN/reasoning.md`, `docs/`	`read, search, edit, execute`	Edits Tom's tests. Starts the next task on his own.



Max MANAGER · MONITOR

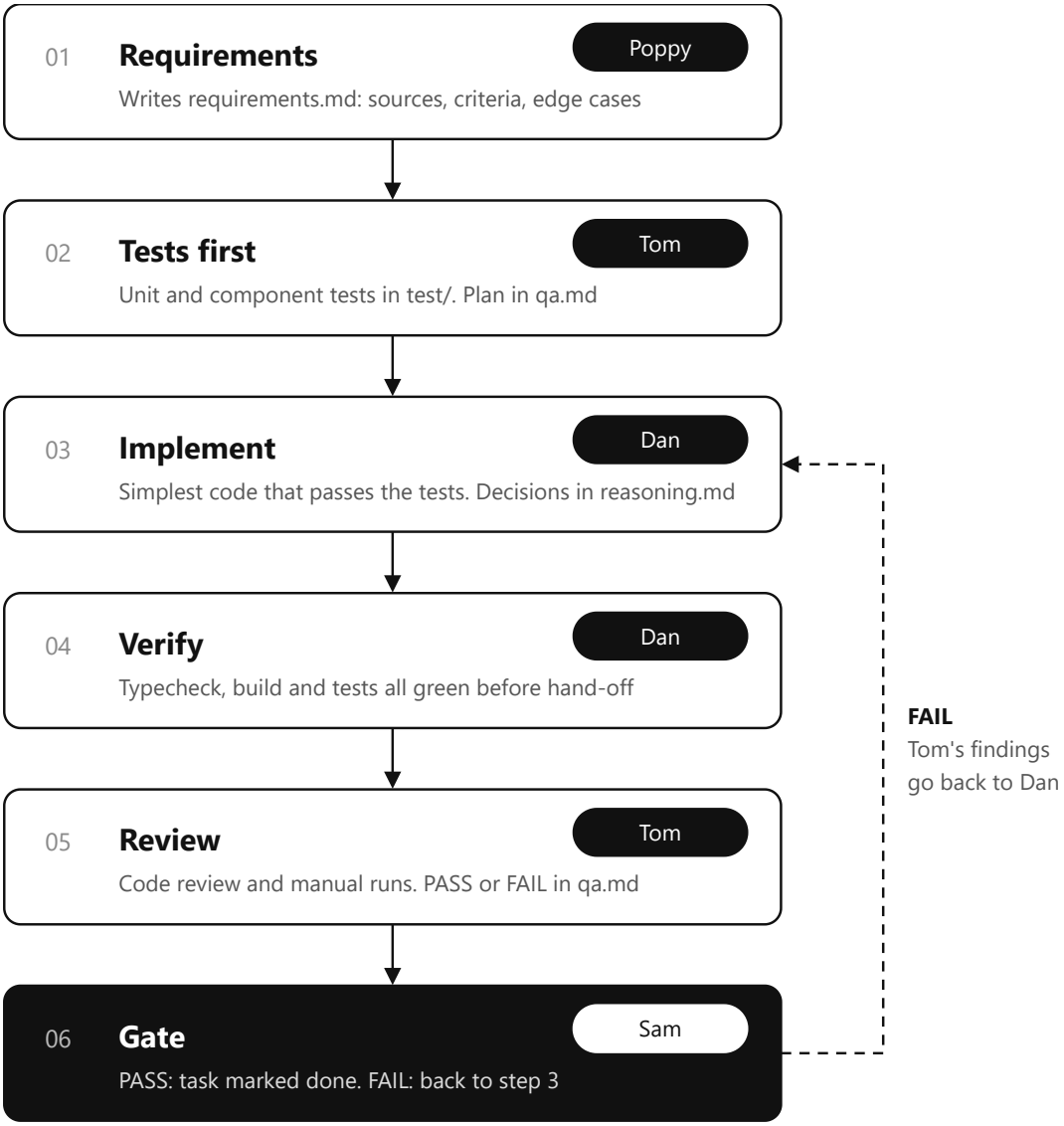
Watches the team. At each task end and each wave end he checks the work folder and the brief, then reports concerns, each with file evidence.

WRITES	TOOLS	NEVER
`work/wave-NN/status.md` only	`read, search, edit`	Implements, tests or decides. Read-only on code.

PROCESS

The task cycle

Every task goes through six steps in order. No step is skipped. The next task starts only after step 6.



After three failed rounds on one task, Sam stops and asks the user.

Rules of the cycle

- Tests come before code. Dan never edits Tom's tests. A wrong test is reported to Sam.
- Poppy writes the requirements from the brief and the assignment. Nothing is invented; every requirement cites its source.
- Max is called at the end of each task and at the end of each wave.
- A wave closes only when every task has PASS, Max raises no open concern, and Tom verifies Dan's hand-off note.
- After the implementation waves pass, the documentation wave runs: Dan writes `docs/` with the service-docs skill, and Tom signs it off.

Files between agents

Agents never share memory across chats. Each step leaves a file, and the next agent reads it. Each row shows who writes a file and who reads it.

FILE OR FOLDER	SAM	POPPY	TOM	DAN	MAX
`work/wave-NN/requirements.md`		W	R	R	R
`test/`			W	R	
`work/wave-NN/qa.md`	R		W	R	R
`src/ and docs/`			R	W	
`work/wave-NN/reasoning.md`			R	W	R
`work/wave-NN/status.md`	R				W

W writes R reads

Wave flow

Waves run in order, one at a time. Tasks inside a wave also run one at a time.

01	02	03	04	05	06
Read the brief	**Open the folder**	**Run the cycle**	**Check**	**Hand off**	**Close**
Sam checks the previous wave is done	Sam creates work/wave-NN/	Each task, in order, six steps	Max reviews each task and the wave	Dan writes the note, Tom verifies it	Next wave starts after this one closes

Escalation

- Sources conflict: stop, write the question in the agent's file and report to Sam. Sam asks the user. Nobody guesses.
- Tom's test looks wrong: Dan reports it to Sam. Dan does not change the test.
- A contract that an earlier wave relies on needs to change: stop and ask.

PLAN

Nine waves

Each wave has a brief in `plan/wave-NN-*/AGENT.md`, with a companion PDF for people. Waves unlock one another in order.

01 02 03 04 05 06 07 08 09

	FOLDER	MISSION
01	`wave-01-foundation`	Project skeleton: package manifest, strict TypeScript, config and health endpoint.
02	`wave-02-building-blocks`	Validation, retry policy and the SQLite schema.
03	`wave-03-adapters`	Four adapters for the outside world and SQLite, each behind a small interface.
04	`wave-04-pipeline`	Snapshot index and the single processing pipeline used by events and CSV rows.
05	`wave-05-entry-points`	Dev message route for single events and the bulk import worker for CSV files.
06	`wave-06-api-and-tests`	Job status endpoint and focused tests for the business rules.
07	`wave-07-e2e`	Full run against the real mocks, with the evidence recorded.
08	`wave-08-documentation`	Solution docs in `docs/`: how to run it, assumptions, decisions and complexity.
09	`wave-09-cleanup`	Removes everything that is not a deliverable.

Rules for everyone

- English only. No emojis. Short, direct writing.
- Do not edit `wiremock/`, `compose.yaml`, `MOCK-ENVIRONMENT.md` or the PDFs.
- Edit files with file tools, not shell redirection.
- Each agent does only its own role.
- Stay inside the current wave scope.
- Blocked or unsure: write it in the file and report to Sam.
- Never commit secrets. No hard-coded URLs outside `src/config.ts`.
- Use the same HTTP client, retry helper and database opener from earlier waves.

Sources

`.github/agents/*.agent.md` · `.github/copilot-instructions.md` · `.github/skills/task-cycle/SKILL.md` · `.github/skills/service-docs/SKILL.md` · `.github/instructions/code-style.instructions.md` · `plan/wave-NN-*/AGENT.md` · `work/wave-NN/`